Lab 2: 时钟中断处理

1 实验目的

以时钟中断为例学习在RISC-V上的异常处理相关机制。

2 实验内容及要求

- 理解RISC-V异常委托与异常处理机制
- 利用OpenSBI平台的SBI调用触发时钟中断，并通过代码设计实现定时触发时钟中断的效果

请各位同学独立完成实验，任何抄袭行为都将使本次实验判为0分。请查看文档尾部附录部分的背景知识介绍，跟随实验步骤完成实验，以截图的方式记录命令行的输入与输出，注意进行适当的代码注释。如有需要，请对每一步的命令以及结果进行必要的解释。

最终提交的实验报告请删除附录后命名为"**学号_姓名_lab2.pdf**"，代码文件请根据给出的结构整理好后打包命名为"**学号_姓名_lab2**"，分开上传至学在浙大平台。

3 实验步骤

3.1 搭建实验环境，理解实验执行流程

请同学们认真阅读**RISC-V中文手册中的【第十章 RV32/64 特权架构】**，对异常及特权模式进行一个系统性的了解，与本实验相关的内容已总结至附录中，可按顺序阅读。

本次实验的目标是**定时触发时钟中断并在相应的中断处理函数中输出相关信息**，最终运行结果可见【3.5 编译及测试】。代码实现逻辑如下：

- 在初始化阶段，设置CSR寄存器以允许S模式的时钟中断发生，利用SBI调用触发第一次时钟中断。
- SBI调用触发时钟中断后，OpenSBI平台自动完成了M模式的中断处理。由于设置了S模式的中断使能，随后触发了S模式下的时钟中断，相关异常处理CSR进行自动转换，需要保存寄存器现场后调用相应的时钟中断处理函数。
- 在时钟中断处理函数中打印相关信息，并设置下一次时钟中断，从而实现定时（每隔一秒执行一次）触发时钟中断并打印相关信息的效果。函数返回后恢复寄存器现场，调用监管者异常返回指令sret回到发生中断的指令。

为了完成实验，需要同学们在`init.c`中设置CSR寄存器允许中断发生，在`clock.c`中触发时钟中断事件，在`entry.S`及`trap.c`中编写中断处理函数。

1. 创建容器，映射文件夹

同lab1的文件夹映射方法，目录名为`lab2`。

```
docker run -it -v
/c/Users/12638/Desktop/operatingSystem/lab/lab2:/home/oslab/lab2 -u oslab -w
/home/oslab fa20 /bin/bash
 docker exec -it  -u oslab -w /home/oslab strange_golick /bin/bash
```

2. 理解组织文件结构

```
kab2
├── arch
|   └── riscv
|       ├── kernel
|       |   ├── clock.c
|       |   ├── head.S
|       |   ├── entry.S
|       |   ├── init.c
|       |   ├── main.c
|       |   ├── Makefile
|       |   ├── print.c
|       |   ├── sbi.c
|       |   ├── trap.c
|       |   └── vmlinux.lds
|       └── Makefile
├── include
|   ├── defs.h
|   ├── riscv.h
|   └── test.h
├── Makefile
└── README.md
```

下载相关代码移动至`lab2`文件夹中，项目代码结构如上所示，**请同学们根据lab1中自己写的代码填写`print.c`以及`sbi.c`中的`sbi_call()`函数。**

附录【B.gef插件】提供了一款gdb插件`gef`的下载安装方式，该插件能够实时显示各个寄存器的值，**方便调试**，有需要的同学可以按照提示进行安装，实验中对此不做要求。

3.修改必要文件

由于裸机程序需要在`.text`段起始位置执行，所以需要利用`vmlinux.lds`中`.text`段的定义来确保`head.S`中的`.text`段被放置在其他`.text`段之前，首先修改`head.S`中的`.text`命名为`.text.init`：

```
<<<<< before
.section .text
============
.section .text.init
>>>>> after
```

修改`entry.S`中的`.text`命名为`.text.entry`：

```
<<<<< before
.section .text
============
.section .text.entry
>>>>> after
```

然后修改`vmlinux.lds`文件中的`.text`展开方式：

```
<<<<< before
.text : {
        *(.text)
        *(.text.*)
     }
============
.text : {
        *(.text.init)
        *(.text.entry)
        *(.text)
        *(.text.*)
     }
>>>>> after
```

3.2 编写init.c中的相关函数（20%）

我们将M模式的事务交由OpenSBI进行处理，从而只需要关注S模式下的处理。为了满足时钟中断在S模式下触发的使能条件，我们需要对以下寄存器进行设置：

1. 设置 `stvec` 寄存器。 `stvec` 寄存器中存储着S模式下发生中断时跳转的地址，我们需要编写相关的中断处理函数，并将地址存入 `stvec` 中。
2. 将 `sie` 寄存器中的stie位 置位。 `sie[stie]` 为1时，s模式下的时钟中断开启。
3. 将 `sstatus` 寄存器中的sie位 置位。 `sstatus[sie]` 位为s模式下的中断总开关，这一位为1时，才能响应中断。

在 `init.c` 中设置CSR寄存器以允许中断发生， `idt_init()` 函数用于设置 `stvec` 寄存器， `intr_enable()` 与 `intr_disable()` 通过设置 `sstatus[sie]` 打开s模式的中断开关。

1.编写 `intr_enable()` / `intr_disable()`

使用CSR命令设置 `sstatus[sie]` 的值，可以使用 `riscv.h` 文件中的相应函数，也可以自行通过内联汇编实现。

```
void intr_enable(void) {
    //设置sstatus[sie]=1 ,打开s模式的中断开关
    //your code
    set_csr(sstatus, 2);
}
void intr_disable(void) {
    //设置sstatus[sie]=0,关闭s模式的中断开关
    //your code
    clear_csr(sstatus, 2);
 }
```

Q1.解释所填写的代码。

答： `set_csr(sstatus, 2);` 设置 sstatus CSR的 sie bit 为1， `clear_csr(sstatus, 2);` 就是把sie bit 清零.

2.编写 `idt_init()`

使用 `riscv.h` 文件中的相应函数向 `stvec` 寄存器中写入中断处理后跳转函数 `trap_s` （entry.S)的**地址**。

```c
void idt_init(void) {
    extern void trap_s(void);
    //向stvec寄存器中写入中断处理后跳转函数的地址
    write_csr(stvec, &trap_s);
}
```

3.3 编写`clock.c`中的相关函数（30%）

我们的"时钟中断"实际上就是"每隔若干个时钟周期执行一次的程序"，可以利用OpenSBI提供的`sbi_set_timer()`接口实现，**向该函数传入一个时刻，函数将在那个时刻触发一次时钟中断**（见`lab1`中对sbi调用的相关说明）。使用SBI调用完成了M模式下的中断触发与处理，不需要考虑`mtime`、`mtimecmp`等相关寄存器。

我们需要"**每隔若干时间就发生一次时钟中断**"，但是OpenSBI提供的接口一次只能设置一个时钟中断事件。我们采用的方式是：一开始只设置一个时钟中断，之后每次发生时钟中断的时候，在相应的中断处理函数中设置下一次的时钟中断。

在`clock.c`中，我们需要定时触发时钟中断事件，`clock_init()`用于启用时钟中断并设置第一个时钟中断；`clock_set_next_event()`用于调用SBI函数`set_sbi_timer()`触发时钟中断。`get_cycle()`函数通过`rdtime`伪指令读取一个叫做`time`的CSR的数值，表示CPU启动之后经过的真实时间。

qemu中外设晶振的频率为10mhz，即每秒钟`time`的值将会增大10^7。我们可以据此来计算每次`time`的增加量，以控制时钟中断的间隔。

XLEN-1 10	9	8	7 6	5	4	3 2	1	0
0	SEIP	UEIP	0	STIP	UTIP	0	SSIP	USIP
XLEN-10	1	1	2	1	1	2	1	1

Figure 4.3: Supervisor interrupt-pending register (`sip`).

XLEN-1 10	9	8	7 6	5	4	3 2	1	0
0	SEIE	UEIE	0	STIE	UTIE	0	SSIE	USIE
XLEN-10	1	1	2	1	1	2	1	1

Figure 4.4: Supervisor interrupt-enable register (`sie`).

https://blog.csdn.net/Pandacooker

Q2.为了使得每次时钟中断的间隔为1s，`timebase`（即`time`的增加量）需要设置为10 000 000__10^7。若需要时钟中断间隔为0.1s，`timebase`需要设置为____10^6 1000 000_。

```c
static uint64_t timebase =1;//需要自行修改timebase为合适的值，使得时钟中断间隔为1s
```

1. 编写`clock_init()`

```c
void clock_init(void) {
    puts("ZJU OS LAB 2              Student_ID:3180103721\n");
    //对sie寄存器中的时钟中断位设置（sie[stie]=1）以启用时钟中断。
    set_csr(sie,32);// stie xlen-1 时 = 第6位
    //设置第一个时钟中断。
    clock_set_next_event();
}
```

2.编写`clock_set_next_event()`

```
void clock_set_next_event(void) {
    //获取当前cpu cycles数并计算下一个时钟中断的发生时刻,通过sbi.c触发时钟中断。
    uint64_t  currentTime =get_cycles();
    trigger_time_interrupt(currentTime+ timebase);
}
```

Q3.解释`sbi.c`中`trigger_time_interrupt()`函数。

答：`sbi_call(0, stime_value, 0, 0);`

sbi_type ==0时, 是调用sbi_set_timer function .

第一个参数`arg0`需要传入 间隔时间 ， arg1-2不需要输入，可以用0填充。

3.4 编写并调用中断处理函数（50%）

1.在`entry.S`中调用中断处理函数（30%）

在【3.1】中，我们向stvec寄存器存入了中断处理函数的地址，中断发生后将自动进行硬件状态转换，程序将读取`stvec`的地址并进行跳转，运行`trap _s`函数。该函数需要在栈中保存`调用者保存寄存器`及`sepc`寄存器，读取`scause`CSR寄存器设置参数`a0`调用`handle_s`函数进行中断处理，调用返回后需要恢复寄存器并使用`sret`命令回到发生中断的指令。

提示：可参考[RISC-V中文手册]3.3节相关内容完成实验；寄存器大小为8字节；CSR命令的特殊使用方式（见lab1附录)；若不清楚`调用者保存寄存器`，可将寄存器全都保存；对汇编语言不是特别了解的建议把中文手册读一遍，用时也不长的。

Q4.为什么要保存`sepc`寄存器：

答： 发生中断的指令的PC 被存入 sepc. 这样可以回到之前的指令

2.在`trap.c`中编写中断处理函数（20%）

正常情况下，异常处理函数需要根据`[m|s]cause`寄存器的值判断异常的种类后分别处理不同类型的异常，但在本次实验中简化为只判断并处理时钟中断。

【3.3】中提到，为了实现"定时触发中断"，我们需要在中断处理函数中设置下一次的时钟中断。此外，为了进行测试，中断处理函数中还需要打印时钟中断发生的次数，需要在`clock.c`中利用`ticks`变量进行统计，请更新【3.3】中相关代码。

```
void handler_s(uint64_t cause){
    // interrupt
    if ( ((cause >> 63) & 1) == 1) {
        // supervisor timer interrupt
        if ( (cause << 1) == 10) {
            //设置下一个时钟中断，打印当前的中断数目。
            clock_set_next_event();
            puts("[S] Supervisor Mode Timer Interrupt ");
            ticks += 1;
            put_num(ticks);
            puts("\n");
        }
    }
    return;
```

```
}
```

Q5.触发时钟中断时，`[m|s]cause`寄存器的值是？据此填写代码中的条件判断语句。

答：中断时mcause 的最高有效位置 1，同步异常时置 0

m寄存器的值是cause=0x8000000000000005).其实看这个来填写就很简单, 一开始我光空想就很难.

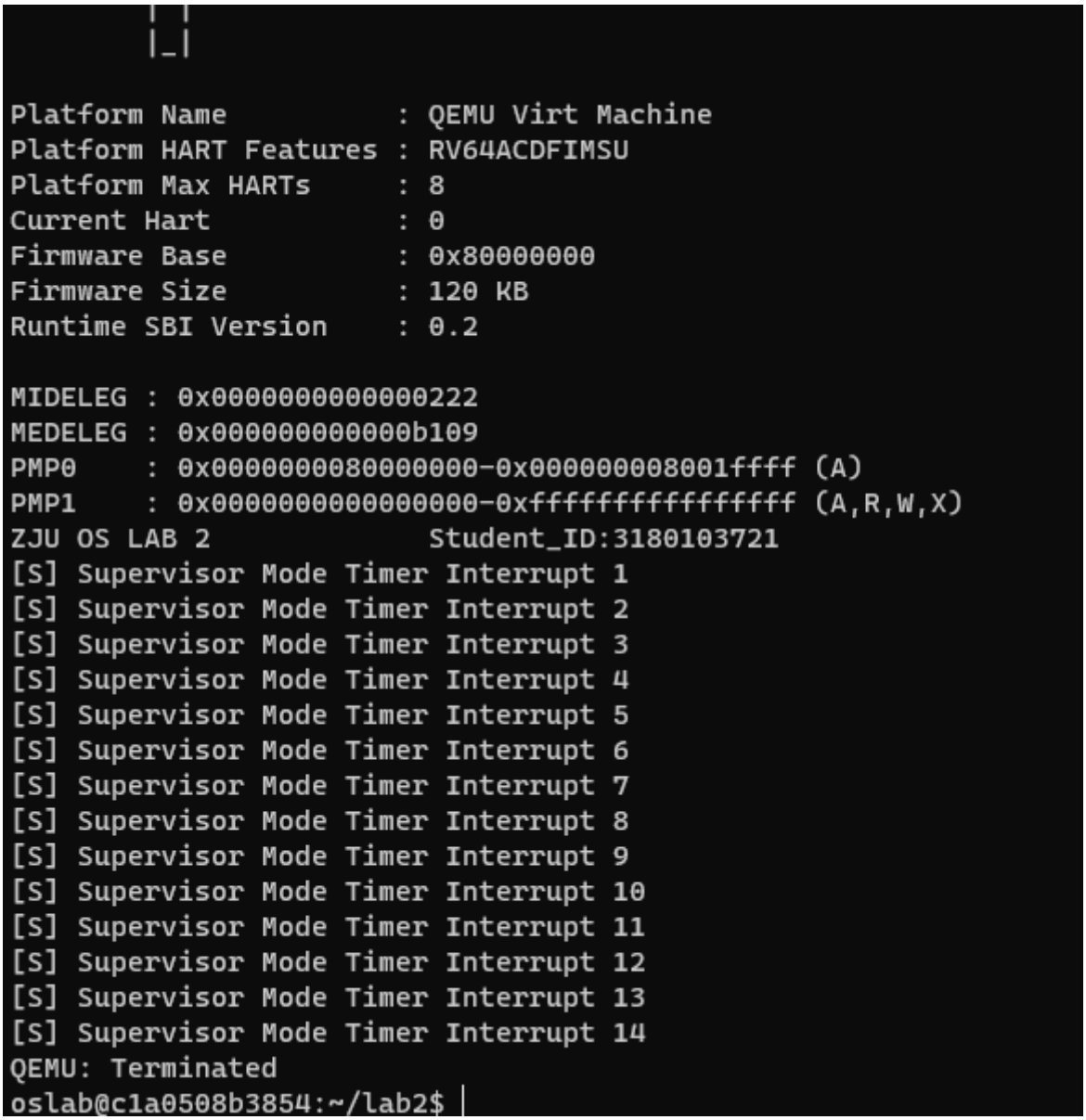
可以仔细看图10.3 supervisor timer interrupt 好像是最高位1 , 后面也是1 . 值应该是 1然后61个0 然后10

Q6.【同步异常与中断的区别】当处理同步异常时应该在退出前给`epc`寄存器+4（一条指令的长度），当处理中断时则不需要，请解释为什么要这样做。

答： mepc储存返回地址。出现同步异常后，返回地址更新为当前发生异常指令的地址，但是真正的返回地址应该是异常指令的下一条指令，故要执行mepc=mepc+4。处理中断时，mepc已储存下一条指令地址，故不需要自增。

3.5 编译及测试

请修改`clock.c`中的`ID:123456`，确保输出自己的学号。仿照`lab1`进行调试，预期的实验结果如下，**每隔一秒触发一次时钟中断，打印一次信息**。请将样例截图替换为自己的截图。



4 讨论和心得

请在此处填写实验过程中遇到的问题及相应的解决方式。

本实验本意是使用OpenSBI平台避免编写过多的汇编代码以简化实验，但是这样做之后省去了实际时钟中断处理时的复杂跳转，把整个过程简化得过于简单了。欢迎同学们对本实验提出建议。

问题1: 怎么往stvec CSR 写入 地址.

可以用csrrw

问题2: entry.S的地址在哪里

write_csr(reg, &trap_s); 不知道这样取地址对不对. 也不知道是否可以直接塞进去. 有一种是先加载到t0. `la t0, trap_entry  csrw stvec, t0`

问题3: 怎么保存寄存器到栈. 栈空间在哪里？哪些是 caller_saved reg

我觉得就存两个, 一个 a0 参数, 一个 spec .

寄存器表可以看[计算机硬件] RISC-V 汇编指令笔记 - GeorgeOkelly - 博客园 (cnblogs.com)

为啥是参考3.3 而不是10.3 , 觉得10.3 讲的很多, 10.6好像就有简单的 RISC-V时钟中断处理程序代码

问题4: 可以make , 但是run会出问题,无法打印时间就死循环了.

调试 make debug

这次没有b start_kernel , 就b *0x80200000 打个断点,

一步步step, 他是先执行init.c 中 intr_enable , 然后clock.c中clock_init . 我应该用next 跳过puts 第一个输出

设clock那边的断点, 但是不知道它的地址. 可以设置函数名 clock_init

stie 右边是5个0, 改成32

因为我set_csr(sstatus, 2); 是10

问题5: (cause=0x0) 没有把cause传入

没有把 cause 保存

我用 t5来读取sepc , 不知道会不会出问题 csrrs t5,sepc,zero

它调用了handler_s, 但是没有 输出. 所以是条件判断出现了问题.

于是我在handler_s打断点 ,看了cause的值, 然后发现条件判断语句有问题, 修改后可以正常输出.